

Fakturama Image-to-Cash — Design

Input: one order image. **Output:** a saved, verified **Order** and a **linked Invoice** with the extracted payment method and paid status. Missing master data (Debtor, terms of payment, VAT, Product) is created through the UI. No Delivery, Correction or Dunning (5.7).

1 Principles

1. **Order first:** the New Order stays open; its selectors are the existence checks.
2. **Fail closed:** any doubt stops for manual review, with reason and evidence.
3. **Verify every step:** read back writes; confirm saves in Data > Documents.
4. **Save exactly once:** never retry blindly, so no duplicates.
5. **No coordinates:** find controls by meaning; pixels only where UIA is blind.
6. **UI only:** the database is never read or written.

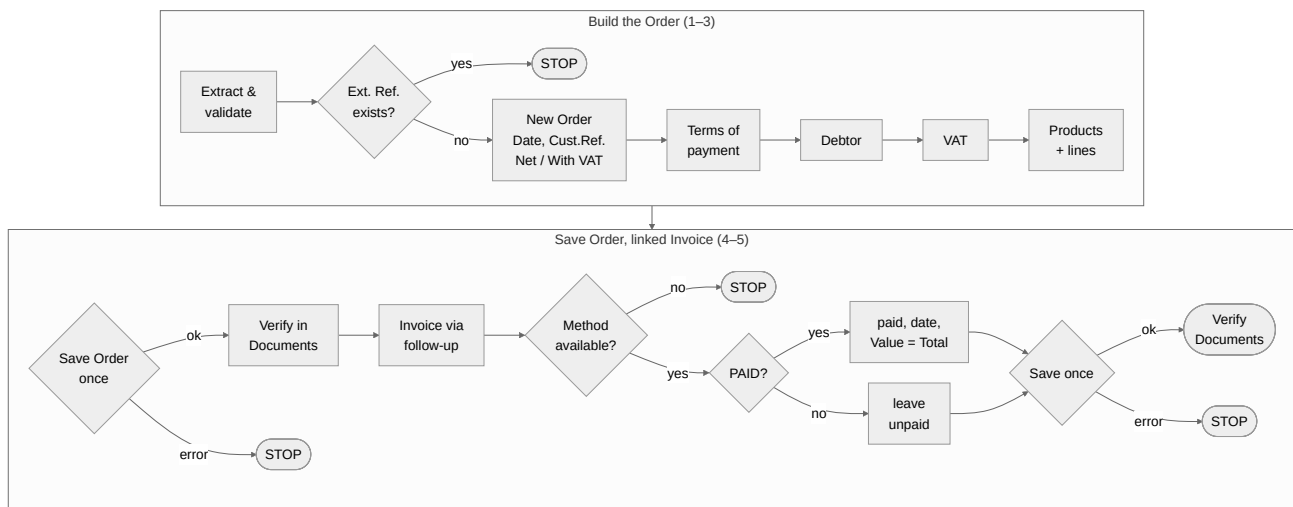


Figure 1: End-to-end Order-first flow.

2 Key assumptions

Topic	Choice
Input	one image, one order, one currency; the image is the only source of truth
Setup	Fakturama 2.x, English UI, initialised workspace; desktop not used during a run
Locale	date format and decimal separator learned from the UI; labels in one file
Contact name	last token = Name (“Marta Klein” → Marta / Klein); Salutation “—”
Customer ID	extracted but unused; Fakturama keeps its own ID
Billing ≠ delivery	Main = Invoice address; extra address = Delivery address
Payment timing	terms of payment run <i>before</i> New Contact (Payment list loads once)
Existing Debtor	methods created only in the Debtor-creation branch; else 5.2 stops
“Exact”	equal ignoring case and spaces; “...” cells match as prefix after widening
Product price	gross = net × (1 + VAT/100), half-up; no line discount
Order level	discount 0%, free shipping; other values fail reconciliation
Paid status	only “PAID” ticks paid (date, Value = Total); else left clear
Re-running	one reference → at most one Order and one Invoice; --resume continues

3 Architecture

Layer	Responsibility
extraction	image $\rightarrow$ validated, typed order
workflow	one stage per PDF section; rules; resume plan
fakturama	page objects for every screen
ui	UIA: find, wait, act, verify, evidence

Rules (matching, pricing, reconciliation, resume) are pure, unit-tested functions. CLI and GUI; options: `--confirm`, `--narrate`, `--resume`, replay, fast mode. Exit codes: 0 done, 2 manual review, 1 verification failed.

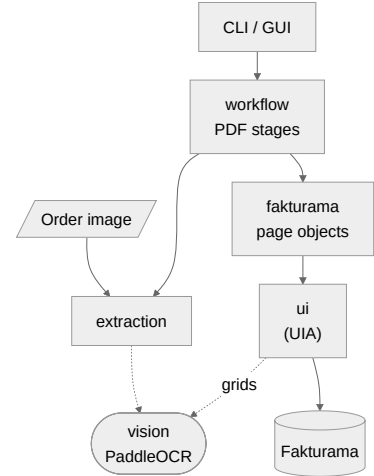


Figure 2: Layers.

4 Image extraction

1. **OCR** (PaddleOCR, local): text, boxes, confidence; small images upscaled.
2. **Layout rows** keep table columns and labels aligned.
3. **LLM** (Groq gpt-oss-120b, text only, temp. 0, strict JSON): copies strings verbatim.
4. **Grounding**: every value in the OCR text, confidence ≥ 0.80 .
5. **Layout check**: each value sits under its own label.
6. **One retry** with a hint naming the wrong field; then stop.
7. **Normalise** to Decimal, ISO dates, %, name and ZIP/city split.
8. **Reconcile**: lines, net, VAT per rate and gross must add up.

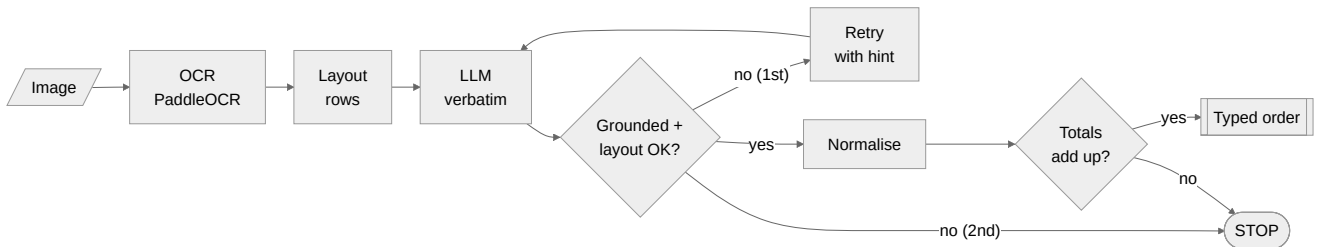


Figure 3: Extraction: the LLM copies, Python checks and computes.

5 Finding controls in Fakturama

Fakturama is Java SWT; controls are found with the strongest signal available:

1. **UIA properties**: name, AutomationId, tooltip.
2. **Label adjacency**: input beside or below its label, from live rectangles.
3. **Structure**: always the *first* icon (never the green +).
4. **Scope**: search only the active editor, dialog or view.
5. **Pixels**: NatTable grids — detect grid lines, OCR each cell separately.

Acting safely. Every setter reads back. Combos by keyboard (UIA only repaints them); dates one segment at a time; keys only into the focused control; mouse wheel only over scrollbars; poll instead of sleeping; a save counts only if the tab is clean with no error.

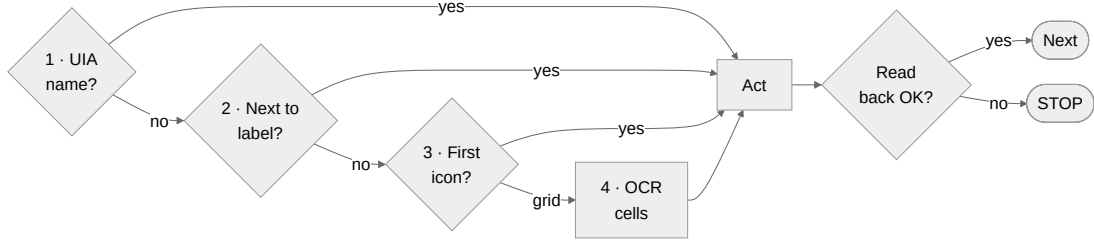


Figure 4: Control discovery and verified action.

6 Matching and manual review

One rule for all lists: **one exact row** → select; **none** → create; **several or conflicting** → review. A just-saved record that does not reappear (2.12, 3.12) stops. An unreadable list never means “missing”.

Record	Identity + exact fields
Debtor	Company + First Name, Name, ZIP, City
Product	SKU + Name
VAT	“VAT n%” + Value, code S
Payment	Name + payment code

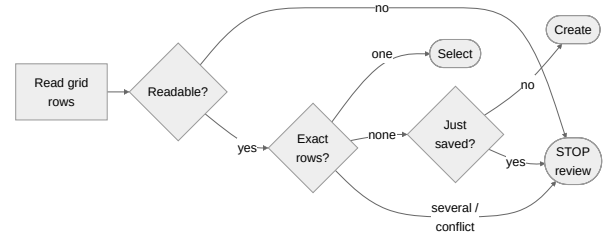


Figure 5: Matching rule.

Manual review stops (exit 2, Fakturama left as is, nothing half-saved): extraction not grounded or not reconciling; reference already exists; conflicting Debtor (2.3), payment method (2.10.2), Product (3.3) or VAT (3.5); a just-saved record not found (2.12, 3.12); Invoice payment method unavailable (5.2); unsaved editors open or a list unreadable.

7 Resume after manual review

--resume decides from **what is saved**, never from a state file, and never creates a second Order or Invoice. Reopened documents match by exact number; Fakturama’s “follow-up already exists” warning is answered *No*. Resuming a finished order saves nothing. *Verified live*: a 5.2 stop (Credit Card missing) resumed into a paid INV000004.

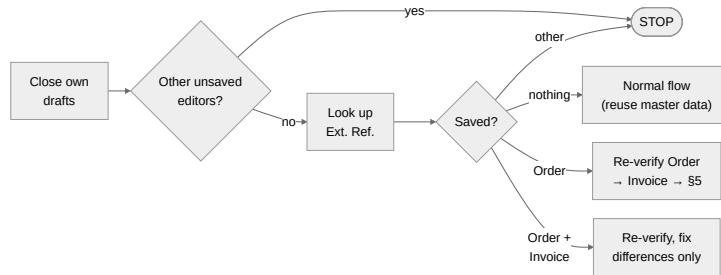


Figure 6: Resume plan from saved documents.

8 Evidence and testing

Evidence: per step a screenshot and a JSON line (values compared, grid rows used); extraction artefacts, `report.json` and a one-page `report.html`. **Tests:** offline unit tests for every rule and grid detection; six generated images (original, degraded, unpaid, same addresses, new product, wrong total); live runs of create, select, 5.2 stop and resume.

9 Fakturama quirks absorbed

- Combo selection only repaints → keyboard plus read-back.
- Mouse wheel changes combos → wheel only over scrollbars.
- Segmented date field → typed one segment at a time.
- Address tabs break an unsaved save → delivery address checked after save.
- Payment list loads once → terms of payment before New Contact.
- Product selector auto-picks one hit → accepted only if the SKU matches.
- Only the visible tab is in UIA → background editors reached by tab title.

10 Trade-offs

Choice	Benefit	Cost
OCR + text LLM	grounded, checkable, cheap	two models; layout rebuilt
LLM copies, Python computes	no LLM math errors	more normalisation code
Fail closed	no wrong financial data	more manual reviews
UIA first, OCR for grids	layout independent	~1 s per grid read
Real mouse/keyboard	SWT gets real events	desktop busy during run
UI-only verification	matches what a user sees	slower; OCR on lists
Resume from saved docs	no stale checkpoints	one extra lookup
Payment before Contact	new method selectable	small PDF reordering

11 Risks and next steps

Risks: OCR noise in lists, shared desktop, untested locales. Next: read NatTable without OCR; run in a dedicated Windows session; test every review branch live; harder images with per-field accuracy metrics.